

Домашнее задание 1

Important notice

- 1) **Это задание выполняется и оценивается индивидуально или в группах до 4 человек!** Если вы используете код из открытых источников (репозитории ваших одноклассников таковыми не считаются) - пожалуйста, указывайте ссылки на них (можно в readme вести лог всех источников, откуда вы берете код). Находить готовые рецепты в интернете - хорошо, списывать - плохо.
- 2) **При обнаружении списывания (одинаковый код в двух репозиториях без указания внешнего источника) оценка будет выставляться студенту (группе), чей коммит с решением был первый.** Остальным - 0 баллов и докладная в УО.
- 3) Чтобы исключить возможность списывания, **рекомендуется сделать ваш репозиторий с домашним заданием приватным.**

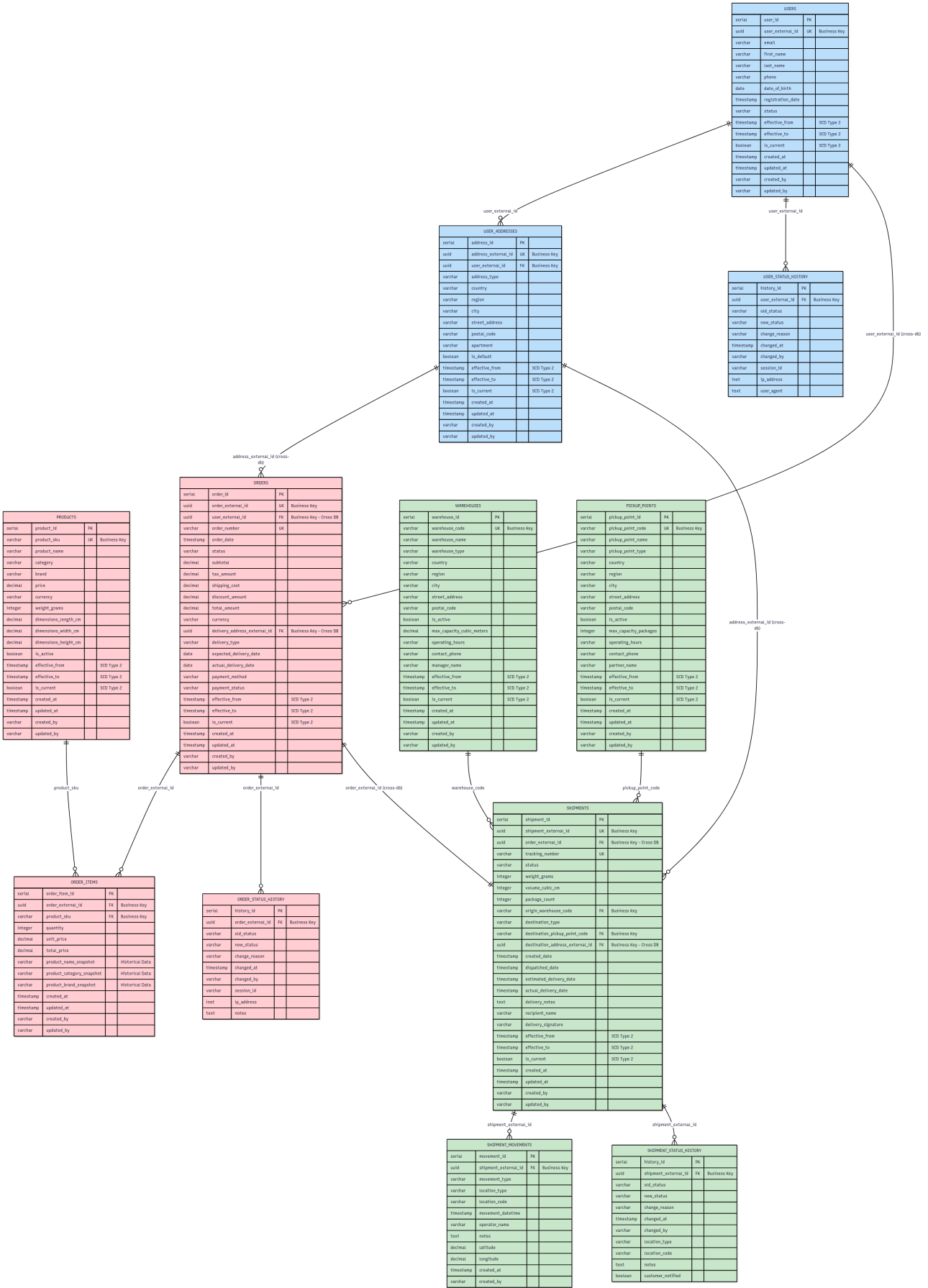
Формулировка

У нас есть 3 микросервиса, которые работают внутри большого маркетплейса:

- **user_service** - содержит информацию о пользователях
- **order_service** - содержит информацию о заказах пользователей
- **logistics_service** - содержит информацию о том, как заказ разбился на посылки и как они перемещаются по логистической сети (от складов к ПВЗ)

Их базы данных выглядят следующим образом:

- **user_service_db** - отмечена синим
- **order_service_db** - отмечена красным
- **logistics_service_db** - отмечена зеленым



Задача:

1. Поднять инстанс PostgreSQL в docker-compose;
2. Создать 3 файла с миграциями, которые создадут структуру трех БД выше (для учебных целей нас утрит, что все эти БД будут жить на одном физическом хосте PostgreSQL); **Важно**, чтобы БД / схемы / таблицы / поля назывались именно так, как указано на схеме - иначе ДЗ не пройдет автотесты;
 1. **user_service_db** -> схема public -> таблицы / constraints / keys
 2. **order_service_db** -> схема public -> таблицы / constraints / keys
 3. **logistics_service_db** -> схема public -> таблицы / constraints / keys
3. Настроить автоматическую инициализацию БД с данной структурой
4. Поднять реплику в docker-compose, настроить репликацию

Бонусные задания:

1. Используя базу данных **order_service_db**, провести **когортный анализ** клиентов для понимания их поведения во времени.
Когорта - это группа клиентов, которые совершили первую покупку в одном временном периоде (месяце).
 1. Определение когорт:
 1. Определить **месяц первой покупки** для каждого клиента (когарту)
 2. Рассчитать **размер каждой когорты** (количество новых клиентов)
 2. Анализ активности когорт:
 1. Для каждой когорты рассчитать количество **активных клиентов** в каждый последующий месяц
 2. Рассчитать **Retention Rate** - долю клиентов, которые вернулись в каждый период
 3. Рассчитать метрики для каждой когорты:
 1. **Total Cohort Revenue**
 2. **Average Revenue Per Customer**
 4. По итогу должна получиться таблица вида:

cohort_month	cohort_size	period_0_pct	period_1_pct	period_2_pct	period_3_pct	period_4_pct	period_5_pct
total_cohort_revenue	avg_revenue_per_customer						
2. Превратить скрипт выше во view
3. Сделать инсталляцию PostgreSQL отказоустойчивой (Patroni, etcd, HAProxy)

Сроки

1. Дедлайн на 100% - 2 недели - **14.10.2025 23:59:59 включительно**
2. Дедлайн на 75% - до дедлайна следующего ДЗ
3. Дедлайн на 50% - до конца курса

Как сдавать ДЗ?

- Готовое ДЗ загружается на GitHub (в приватный репо + выдать мне доступ - @mgcrp)
- ДЗ №1 выполняется в ветке HW 1
- Загружайте только файлы конфигурации, код и markdown с описанием; не загружайте в GitHub бинари, архивы с кодом, видео и/или файлы pgdata; Для того, чтобы такие файлы не попадали в репозиторий, настройте себе .gitignore
- К репо должен быть приложен README.md с описанием того, что вы сделали и как это запустить
 - **Обязательно** должен быть указан connection_string к вашей БД или балансеру
- Задание сдается в форму: <https://forms.gle/pMueh5Jnqt4oWFzt6>

Критерии оценки

Балл	Критерий
4	Запускается только master (с помощью docker-compose), БД инициализирована с нужной структурой
8	Запускаются master и replica (с помощью docker-compose), обе БД инициализированы с нужной структурой, работает репликация
+2 балла	Написан SQL-скрипт, который считает витрину для когортного анализа
+1 балл	Скрипт из предыдущего пункта преобразован во view
+2 балла	Настроен HA-кластер PostgreSQL

Максимальный балл за ДЗ - 13

Как это будет проверяться?

- 1) Запуск проекта по инструкции из вашего readme.md
 - docker-compose up -d или bash-скрипт (если вам нужно сделать несколько действий сразу)
- 2) Проверка количества хостов
- 3) Проверка того, что replication slot активен, на мастер идет streaming, а на replica - receiving
- 4) Проверка структуры БД на хостах
- 5) **Автотест:** python-скрипт, который создает синтетические данные и пишет их в master, после проверяет, что все сгенеренные данные попали в реплику
- 6) Проверка бонусных скриптов и view на загруженных в базу синтетических данных